Actividad 1.1.3:Análisis Comparativo de Algoritmos de Ordenamiento

CREACIÓN DE ARREGLOS EN MEMORIA GLOBAL

```
import random

A = [random.randint(1, 10) for i in range(10)]
A_up = list(range(1, 11))
A_down = list(range(10, 0, -1))

B = [random.randint(1, 100) for i in range(100)]
B_up = list(range(1, 101))
B_down = list(range(100, 0, -1))

C = [random.randint(1, 1000) for i in range(1000)]
C_up = list(range(1, 1001))
C_down = list(range(1000, 0, -1))

D = [random.randint(1, 10000) for i in range(10000)]
D_up = list(range(1, 10001))
D_down = list(range(10000, 0, -1))

arreglos = [
    ("A (10)", A, A_up, A_down),
    ("B (100)", B, B_up, B_down),
    ("C (1000)", C, C_up, C_down),
    ("D (10000)", D, D_up, D_down)
]

print("Muestra de los arreglos generados:")
for nombre, original, up, down in arreglos:
    print(f"\n--- Tamaño: {nombre} ---")
    print(f"Desordenado: {original}")
    print(f"Ascendente:  {up}")
    print(f"Descendente: {down}")
```

```
Muestra de los arreglos generados:

--- Tamaño: A (10) ---
Desordenado: [3, 10, 3, 5, 7, 3, 9, 10, 3, 8]
Ascendente:  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Descendente: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

--- Tamaño: B (100) ---
Desordenado: [34, 68, 20, 8, 65, 36, 10, 83, 42, 40, 13, 19, 83, 62, 77, 2, 34, 5, 75, 14, 6, 20, 48, 41, 1, 41, 69,
Ascendente:  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
Descendente: [100, 99, 98, 97, 96, 95, 94, 93, 92, 91, 90, 89, 88, 87, 86, 85, 84, 83, 82, 81, 80, 79, 78, 77, 76, 75

--- Tamaño: C (1000) ---
Desordenado: [186, 429, 179, 727, 594, 921, 188, 68, 322, 869, 259, 841, 101, 302, 949, 594, 572, 579, 959, 421, 140,
Ascendente:  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
Descendente: [1000, 999, 998, 997, 996, 995, 994, 993, 992, 991, 990, 989, 988, 987, 986, 985, 984, 983, 982, 981, 98

--- Tamaño: D (10000) ---
Desordenado: [9415, 7288, 3500, 2896, 5302, 7633, 8258, 3607, 6776, 6537, 4418, 6864, 7715, 8999, 7086, 6903, 3056, 6
Ascendente:  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
Descendente: [10000, 9999, 9998, 9997, 9996, 9995, 9994, 9993, 9992, 9991, 9990, 9989, 9988, 9987, 9986, 9985, 9984,
```

```
Start coding or generate with AI.
```

Insertion sort algorithm

```
def insertion_sort(array):
    comparaciones = 0

    for i in range(1, len(array)):
        key = array[i]
        j = i - 1
```

```
        while j >= 0:
            comparaciones += 1

            if array[j] > key:
                array[j + 1] = array[j]
                j -= 1
            else:
                break

        array[j + 1] = key

    return comparaciones



print("--- RESULTADOS INSERTION SORT ---")
for nombre, original, up, down in arreglos:
    print(f"\nEvaluando tamaño: {nombre}")

    comp_aleatorio = insertion_sort(original.copy())
    print(f"Desordenado: {comp_aleatorio} comparaciones")

    comp_up = insertion_sort(up.copy())
    print(f"Ascendente:  {comp_up} comparaciones")

    comp_down = insertion_sort(down.copy())
    print(f"Descendente: {comp_down} comparaciones")
```

```
--- RESULTADOS INSERTION SORT ---

Evaluando tamaño: A (10)
Desordenado: 24 comparaciones
Ascendente:  9 comparaciones
Descendente: 45 comparaciones

Evaluando tamaño: B (100)
Desordenado: 2165 comparaciones
Ascendente:  99 comparaciones
Descendente: 4950 comparaciones

Evaluando tamaño: C (1000)
Desordenado: 254713 comparaciones
Ascendente:  999 comparaciones
Descendente: 499500 comparaciones

Evaluando tamaño: D (10000)
Desordenado: 25149828 comparaciones
Ascendente:  9999 comparaciones
Descendente: 49995000 comparaciones
```

Bubble sort

```
def bubble_sort(array):
    comparaciones = 0
    n = len(array)

    for i in range(0, n - 1):
        intercambiado = False

        for j in range(n - 1, i, -1):
            comparaciones += 1
            if array[j] < array[j - 1]:
                array[j], array[j - 1] = array[j - 1], array[j]
                intercambiado = True

        if not intercambiado:
            break
    return comparaciones



print("--- RESULTADOS BUBBLE SORT ---")
for nombre, original, up, down in arreglos:
    print(f"\nEvaluando tamaño: {nombre}")

    comp_aleatorio = bubble_sort(original.copy())
    print(f"Desordenado: {comp_aleatorio} comparaciones")
```

```
    comp_up = bubble_sort(up.copy())
    print(f"Ascendente:  {comp_up} comparaciones")

    comp_down = bubble_sort(down.copy())
    print(f"Descendente: {comp_down} comparaciones")
```

```
--- RESULTADOS BUBBLE SORT ---

Evaluando tamaño: A (10)
Desordenado: 44 comparaciones
Ascendente:  9 comparaciones
Descendente: 45 comparaciones

Evaluando tamaño: B (100)
Desordenado: 4740 comparaciones
Ascendente:  99 comparaciones
Descendente: 4950 comparaciones

Evaluando tamaño: C (1000)
Desordenado: 497484 comparaciones
Ascendente:  999 comparaciones
Descendente: 499500 comparaciones

Evaluando tamaño: D (10000)
Desordenado: 49981797 comparaciones
Ascendente:  9999 comparaciones
Descendente: 49995000 comparaciones
```

Merge Sort

```
def merge(A, p, q, r, comparaciones):
    n1 = q - p + 1
    n2 = r - q

    L = [0] * n1
    R = [0] * n2

    for i in range(n1):
        L[i] = A[p + i]

    for j in range(n2):
        R[j] = A[q + 1 + j]

    i = 0
    j = 0

    for k in range(p, r + 1):
        if i < n1 and j < n2:
            comparaciones += 1
            if L[i] <= R[j]:
                A[k] = L[i]
                i += 1
            else:
                A[k] = R[j]
                j += 1

        elif i >= n1:
            A[k] = R[j]
            j += 1
        else:
            A[k] = L[i]
            i += 1

    return comparaciones

def merge_sort(A, p, r, comparaciones=0):
    if p < r:
        q = (p + r) // 2
        comparaciones = merge_sort(A, p, q, comparaciones)
        comparaciones = merge_sort(A, q + 1, r, comparaciones)
        comparaciones = merge(A, p, q, r, comparaciones)

    return comparaciones
```

```
print("--- RESULTADOS MERGE SORT ---")
for nombre, original, up, down in arreglos:
    print(f"\nEvaluando tamaño: {nombre}")

    comp_aleatorio = merge_sort(original.copy(), 0, len(original) - 1)
    print(f"Desordenado: {comp_aleatorio} comparaciones")

    comp_up = merge_sort(up.copy(), 0, len(up) - 1)
    print(f"Ascendente:  {comp_up} comparaciones")

    comp_down = merge_sort(down.copy(), 0, len(down) - 1)
    print(f"Descendente: {comp_down} comparaciones")
```

Show hidden output